Ders 9 Genişletilmiş Notu: İnterpolasyon Araması, Debug, Arama Algoritmalarının Karşılaştırması, Merge ve Lab-1

Ders 9 Genişletilmiş Notu: İnterpolasyon Araması, Debug, Arama Algoritmalarının Karşılaştırması, Merge ve Lab-1

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Girişi: Alternatif Çözümler ve Derleyerek İlerlemek
- 2. R Basamaklı Palindromlar: Yazdırmak ile Saymak Farklı Problemlerdir
- 3. İnterpolasyon Aramasının Fikri: Ortaya Ateş Etmek Yerine Nişan Almak
- 4. İkili Aramayı do-while ile Yazma: İlk Deneme
- 5. İnterpolasyon Aramasını Kodlama Denemesi
- 6. İkili Aramanın do-while Sürümü
- 7. Dev-C++'ta Hata Ayıklama (Debug)
- 8. Hangi Arama Ne Zaman Kullanılır?
- 9. İki Sıralı Diziyi Birleştirme (Merge)
- 10. Merge Algoritmasının C Kodu
- 11. Değişkenlerin Tanımlanma Yeri
- 12. Arama Algoritmalarının Toplu Karşılaştırması
- 13. Birinci Laboratuvar Sorusu: Ardışıklığı Bozan Sayı

1. Dersin Girişi: Alternatif Çözümler ve Derleyerek İlerlemek

Ders, laboratuvar sorusunun çözüm yaklaşımlarına dair sorularla başladı. Sorunun metni bu notun 13. bölümündedir. Kayıtta sorunun anlatımı en sonda yer alır, ama kayıt karelerindeki saatlere göre laboratuvar bu dersten önce yapılmıştır. Bu yüzden öğrenciler daha ilk dakikalarda soruyu biliyordu.

1.1 Toplam Farkıyla Kayıp Sayıyı Bulmak

Soru: Laboratuvar sorusu bit düzeyinde işlemlerle (bitwise) çözülebilir miydi?

Cevap: Hoca, olabileceğini ve farklı yaklaşımlar yazılabileceğini söyledi. Algoritma verimliliğine dair "olabilir" demekle yetindi, çünkü bu çözümler üzerinde kendisi düşünmemişti.

Soru: Ardışık sayıların olması gereken toplamından girilen sayıların toplamı çıkarılırsa kayıp sayı bulunmaz mı?

Cevap: Hoca bunun olabileceğini söyledi. Dizi birer birer arttığı için, 3'ten başlasa da -3'ten $+8$'e gitse de son terim ve ilk terimden toplam formülüyle beklenen toplam bulunabilir. Sorunun hazırlanış fikri ikili arama (binary search) mantığıydı. İkili arama logaritmik bir algoritmadır. Öğrencinin önerdiği yöntem ise hocaya göre daha çok "sabit" (constant) bir algoritmaya benziyordu ve daha hızlı çözüyor gibiydi. Hoca bir uyarı ekledi: Burada yalnızca verimlilik (efficiency) değil doğruluk da önemlidir. Bu yöntem doğru sonucu veriyorsa daha iyi bir algoritma olur ve kabul edilir. Hoca öneriyi not alıp asistanlara iletti.

Laboratuvarın birinci örneğinde bu yöntem şöyle çalışır (ara adım eklendi). Dizi 2 3 4 5 7 8 9 10'dur:

- Olması gereken: 2'den 10'a kadar 9 sayının toplamı $\frac{(2+10)\cdot 9}{2} = 54$.
- Girilen sayıların toplamı: $2+3+4+5+7+8+9+10 = 48$.
- Fark: $54 - 48 = 6$. Kayıp sayı 6'dır.

  Ders sonrası not: Hoca bu yöntemin sabit bir algoritmaya benzediğini söyledi. Dizinin n elemanının her biri toplama girdiği için yöntem büyük olasılıkla sabit değil, n adımlıdır. Ayrıca hocanın doğruluk uyarısı burada önemlidir. Laboratuvarda birden fazla sayı kayıp olabiliyordu (13. bölüm). Laboratuvar anlatımındaki "8 olmasaydı" durumunda dizi 2 3 4 5 7 9 10 olur. 6 ve 8 kayıpken fark $54 - 40 = 14$ çıkar. Bu değer tek bir kayıp sayı değildir (python ile doğrulandı).

1.2 Kodu Derleye Derleye Yazmak

Bir öğrenci, kodunun bir satırında fazladan bir karakter olduğunu son dakikada fark ettiğini anlattı. Hoca bunun öğrencilerin çok yaptığı bir hata olduğunu söyledi: Kodu sonuna kadar hiç çalıştırmadan yazmak.

> **Hocanın vurgusu:** Kodu derleye derleye gidin. Yaptığınız şeyde hata var mı yok mu, her adımda derleyin. Bu, o ana kadar yaptığınız hataları görmenizi sağlar.

Hoca bunu ikili aramaya benzetti: Programın tamamında hata aramak yerine, yalnızca o ana kadar yazdığınız kısımda hata var mı diye bakarsınız. Böylece arama alanı küçülür.

2. R Basamaklı Palindromlar: Yazdırmak ile Saymak Farklı Problemlerdir

Geçen haftanın konusunda bir yanlış anlaşılma olduğu için hoca palindrom sayıları kısaca tekrar etti. Önemli olan ayrım şudur: "R basamaklı palindrom sayıları ekrana yazın" ile "R basamaklı palindrom sayıların sayısını bulun" farklı problemlerdir ve farklı çözümler gerektirir.

2.1 Palindromları Yazdırmak

Basamak sayısı sabitse sayı doğrudan üretilebilir. Örneğin 6 basamaklı bir palindrom ABCCBA biçimindedir. Üç iç içe döngüyle A, B ve C seçilir, sayının ikinci yarısı bunların tersten yazılmasıyla elde edilir. 7 basamak olsaydı araya bir döngü daha eklenirdi. Ama R genel bir değişkense bu yol tıkanır: Program çalışırken R tane iç içe döngü üretecek bir yapı yoktur. Bu ancak farklı programlama teknikleriyle yapılır ve hoca bunun belki iki sene sonra görülecek bir konu olabileceğini söyledi.

Genel çözüm tek bir döngüdür. R basamaklı ilk sayıdan son sayıya kadar gidilir. Geçen derste görülen sayının tersini bulma algoritmasıyla her sayı ters çevrilir ve kendisiyle karşılaştırılır. Eşitse sayı palindromdur.

> **Ders sonrası not:** Hoca ilk sayıyı 10^R, son sayıyı $10^{R+1}-1$ olarak söyledi. Doğrusu, R basamaklı ilk sayı 10^{R-1}, son sayı $10^R - 1$'dir. Örneğin $R = 3$ için aralık 100 ile 999'dur (python ile doğrulandı).

Genel çözüm her zaman çalışır, ama yavaştır. Basamak sayısı biliniyorsa üretme yöntemi çok daha hızlıdır. 6 basamak için genel çözümde $999999 - 100000 + 1 = 900000$ tur dönülür, ABCCBA üretiminde ise $9 \cdot 10 \cdot 10 = 900$ tur (ara adım eklendi).

2.2 Palindromları Saymak

Hoca sayma problemini basamak basamak kurdu. Bir basamağa gelen rakam 10 çeşittir, ama en baştaki basamağa 0 yazılamaz. Palindromda karşılıklı basamaklar eşit olduğu için, yarıdaki bir basamak seçilince karşılığı da belirlenmiş olur.

Basamak	Biçim	Serbest seçimler	Sayı
1	A	9	9
2	AA	9	9
3	ABA	$9 \cdot 10$	90
4	ABBA	$9 \cdot 10$	90
5	ABCBA	$9 \cdot 10 \cdot 10$	900
6	ABCCBA	$9 \cdot 10 \cdot 10$	900

Hocanın kullandığı bu düzenin genel formülü aşağıdadır. Kesir tam sayı bölmesi olarak, yani aşağı yuvarlanarak alınır:

$$9 \times 10^{\lfloor (R-1)/2 \rfloor}$$

Bütün değerler 1'den 6'ya kadar her R için tek tek sayılarak python ile doğrulandı. Hoca bu formülün biraz permütasyon ve kombinasyon bilgisi gerektirdiğini, kafa karışıklığının buradan çıktığını söyledi.

Aynı sayıyı bulmak için ilk sayıdan son sayıya her sayının palindrom olup olmadığına bakıp bir sayaçla saymak da bir çözümdür. Ama bu mertebesi 10^R olan bir döngü demektir ve iş çok uzun sürer. Formülde ise yalnızca bir üs alma ve bir çarpma vardır. Üs, bir döngüyle üs kadar adımda hesaplanır. Hoca $R = 20$ için bunu kabaca "10 defa döngü, bir de çarpma, 14-15 adımda biter" diye özetledi.

> **Ders sonrası not:** Hoca $R = 20$ için tek tek kontrolde döngü sayısını önce "9×10^{20} gibi", sonra "yüz binler mertebesinde" olarak andı. Tam değer 9×10^{19} turdur. Bu da yüz binlerin çok üzerindedir. Formüldeki üs ise $\lfloor 19/2 \rfloor = 9$'dur (python ile doğrulandı).

3. İnterpolasyon Aramasının Fikri: Ortaya Ateş Etmek Yerine Nişan Almak

Hoca interpolasyon aramasını (interpolation search) Sayısal Analiz dersindeki bir yöntemden yola çıkarak anlattı. Yarılama (bisection) yönteminde yeni nokta $\frac{a+b}{2}$ ile bulunur. Bu, aralığı $\frac{1}{2}$ gibi bir katsayıyla çarpmak demektir. Akla şu soru gelir: $\frac{1}{2}$ yerine $\frac{x}{y}$ gibi bir katsayı alırsak daha iyi bir sonuca ulaşabilir miyiz? Bu bir tür izdüşümdür (projeksiyon).

Hoca bunu hedef vurmaya benzetti. Yarılama yönteminde hedefin solunu vurursunuz, sonra sağını, sonra ortasını, sürekli ortalaya ortalaya gidersiniz. Bu yorucu bir iştir. Regula falsi yani doğrusal interpolasyon yöntemi ise nişan alıp dürbünle bakmak gibidir. Sayısal Analiz'de çıkarılan formül şuydu:

$$c = \frac{b\,f(a) - a\,f(b)}{f(a) - f(b)}$$

İkili aramadaki $\frac{1}{2}$ mantığından esinlenerek aynı şey dizide yapılır: Yalnızca orta nokta hesabı değişir, algoritmanın gerisi ikili aramayla aynıdır. İkili aramada olduğu gibi `low` sıfırıncı elemandan, `high` son elemandan başlar.

Hoca önemli bir farka da dikkat çekti. Sayısal Analiz'de a ve b süreklidir, aradaki ondalıklı sayılar da işe katılır. Dizide ise işler indisler üzerinden yürür. Bu yüzden hesaplanan değer bir şekilde tam sayıya dönüştürülmelidir.

4. İkili Aramayı do-while ile Yazma: İlk Deneme

Bir öğrenci geçen derste ikili aramanın `do-while` ile yazılıp yazılamayacağını sormuştu. Hoca "olur" dedi. Gerekçe, `mid` hesabının döngüden önce ve döngü içinde iki kez yazılmasına gerek kalmamasıdır.

Hoca kodu ekranda dönüştürdü: `while` satırını aşağı aldı, orta nokta hesabını `do` bloğunun içine taşıdı. Ama bir de `key` ile orta noktadaki elemanın karşılaştırılması vardır ve bu da aynı blokta yer alır. Hoca "acaba bir iterasyon fazla mı çalışacak" diye emin olamadığını söyleyip kodu çalıştırdı.

Program 14'ü ve 88'i arıyordu. Dizide 14 vardır, 88 yoktur. Sonuç şaşırtıcıydı: 14 bulunamamış, 88 bulunmuş görünüyordu. Hoca bunun üzerinde tekrar düşünmek gerektiğini söyleyip kodu eski hâline aldı ve interpolasyon aramasına geçti. Doğru çalışan `do-while` sürümü dersin ilerisinde yazıldı (6. bölüm).

5. İnterpolasyon Aramasını Kodlama Denemesi

Bu bölüm dersin yaşanan akışıdır. Hoca formülü canlı olarak kodlamaya çalıştı, kod çalışmadı ve hata birkaç adımda arandı. Sonucun yanında bu süreç de öğreticidir, çünkü debug ihtiyacını gösterir.

Derste kullanılan dizi, kayıt karesine göre `{20, 54, 7, 8, 9, 11, 13, 14, 18, 19}` idi. Sıralandıktan sonraki hâli şöyledir:

İndis	0	1	2	3	4	5	6	7	8	9
Değer	7	8	9	11	13	14	18	19	20	54

5.1 İlk Yazım

Hoca `mid` değerini önce -1 olarak başlattı. Sonra kağıttaki notundan formülü şu kalıpta yazdı: `low` artı, `high - low` farkının dizideki değer farkına bölümü, çarpı `key` ile `a[low]` farkı. Tam sayı bölmesi sıfıra yuvarlanmasın diye de `double` tip dönüşümü (casting) kullandı. Çıktıda iki aranan için de -1 döndü. Bir denemede program dizinin dışına taştı.

> **Ders sonrası not:** Dizinin dışına taşma büyük olasılıkla şöyle oluşmuştur. 88 aranırken $mid = 0 + \frac{9}{54-7} \cdot (88 - 7) = 15{,}51$ olur, tam sayıya dönünce 15'tir. Dizinin son indisi 9'dur. Aranan değer `a[low]` ile `a[high]` arasında değilse formül dizinin dışını gösterebilir (python ile doğrulandı).

5.2 Regula Falsi Formülünü Doğrudan Yazmak ve Debug

Hoca sonra slayttaki formülü harfi harfine koda aktarmayı denedi: b yerine `high`, a yerine `low`, $f(a)$ yerine `a[low]`, $f(b)$ yerine `a[high]`. Tam sayı çarpımları tam sayı verdiği ve bölme sıfıra düşebileceği için ifadeyi `1.0 * high` ile başlatıp her şeyin `double` üzerinden gitmesini sağladı, en sonda da sonucu `int` yaptı. Ayrıca `mid`'in başlangıcını -1 yerine aynı formülle değiştirdi, çünkü "ilk seferde bulma imkânı" vardır.

Bu kez program sonsuz döngüye girdi. Hoca debug'a geçti ve değişkenleri "Add watch" ile izledi. Karede görülen değerler şunlardır:

İzlenen ifade	Değer
`mid`	-1
`low`	0
`high`	9
`a[low]`	7
`a[high]`	54
`(1.0*high)*a[low]-low*a[high]`	63
`a[low]-a[high]`	-47

Hoca "63 bölü eksi 47" dedi ve paydanın daima negatif geleceğine dikkat çekti. Sıralı dizide `a[low] - a[high]` her zaman eksidir. Bir öğrenci de doğru yere işaret etti: Formül normalde $f(a)$ eksi olduğu için uygulanıyordu, ama burada $f(a)$ eksi değildi. Hoca "olabilir" dedi. Başka

bir öğrenci sorunun dizinin süreksiz olmasından kaynaklanıp kaynaklanmadığını sordu. Hoca, dizinin de sürekli gibi düşünülebileceğini söyledi.

Formülün neden yanlış değer ürettiği şu hesapla görülür:

$$mid = \frac{9 \cdot 7 - 0 \cdot 54}{7 - 54} = \frac{63}{-47} \approx -1{,}34 \;\Rightarrow\; -1$$

Ders sonrası not: Derste formül çalıştırılamadı, hata sebebi de sonuna kadar gösterilmedi. Sebep büyük olasılıkla şudur: Regula falsi bir fonksiyonun sıfırını arar. Dizide `key` arandığı için fonksiyon $f(i) = a[i] - key$ olmalıdır, yani formülde `a[low]` yerine `a[low] - key`, `a[high]` yerine `a[high] - key` gelir. 14 aranırken bu, $mid = \frac{9\cdot(7-14)-0\cdot(54-14)}{(7-14)-(54-14)} = \frac{-63}{-47} \approx 1{,}34$ yani 1 verir. Aynı yerine koyma sadeleşince birazdan görülecek hazır koddaki formül çıkar: $mid = low + \frac{(key-a[low])(high-low)}{a[high]-a[low]}$. Yanlış formülde $mid = -1$ olunca `a[-1]` dizinin dışıdır. O değer `key`'den küçük çıkarsa `low = mid + 1 = 0` olur, durum hiç değişmez ve büyük olasılıkla dersteki sonsuz döngü böyle oluşur (sayısal değerler python ile doğrulandı).

5.3 Hazır Kodla Karşılaştırma

Hoca internette bir örnek aradı ve TutorialsPoint'teki C kodunu açtı. O koddaki orta nokta hesabı, kayıt karesine göre şuydu:

```
int mid = lo + (((double)(hi - lo) / (list[hi] - list[lo]))
                * (data - list[lo]));
```

Yani indis farkı değer farkına bölünür, sonuç aranan değerin başlangıçtan uzaklığıyla çarpılır ve `lo`'ya eklenir. Hoca kendi notunun da buna benzer olduğunu söyledi. Örnek kod çalıştırıldığında aranan eleman bir denemede bulundu.

5.4 break Kullanmadan Yazmak

Örnek kodun döngüsünde `break` vardı: Eleman bulununca ayrı bir `index` değişkenine atanıp döngüden çıkılıyordu. Hoca bunu kullanmak istemediğini söyleyerek kodu düzenledi. Düzenleme sırasında bir süre eleman bulunamadı. Bir öğrencinin de dikkat çektiği gibi `index` değişkeni `mid`'e eşitlenmemişti. Hoca çözümü sadeleştirdi: Bulunuyorsa doğrudan `mid` gönderilir, `index`'e hiç gerek yoktur. Karedeki döngü bu düzenlemeyle şu yapıdadır (yazdırma satırları atlandı):

```
while (lo <= hi && list[mid] != data) {
    comparisons++;
    mid = lo + (((double)(hi - lo) / (list[hi] - list[lo]))
                * (data - list[lo]));
    if (list[mid] < data) {
        lo = mid + 1;
    } else {
        hi = mid - 1;
    }
}
```

`mid` döngüden önce interpolasyon formülüyle başlatılır. Eleman ilk atışta bulunmuşsa döngüye hiç girilmez. Bulunamamışsa döngü devam eder.

Hoca, `break`'li hâlin hem ekstra değişken sakladığını hem de gereksiz bir dolambaç içerdiğini söyledi ve bir benzetme yaptı: "Sağ kulağınızı tutun" denince kolunuzu başınızın arkasından iki kez dolaştırıp sol elinizle tutmak gibidir. Burada ekstra işler yapılmıştır, hâlbuki gerek yoktur.

Hoca kendi Dev-C++ kodunda formül üzerinde ufak değişiklikler yapması gerektiğini, formülü tekrar inceleyeceğini ve kodu paylaşacağını söyledi.

Ders sonrası not: Ders sonrasında dersteki diziyle çalışan bir sürüm yazıldı ve `gcc -Wall -Wextra -std=c89 -pedantic` ile uyarısız derlendi. Sürüm, karedeki yapıya üç şey ekler: `key` aralık dışındaysa döngüye girilmez, `a[high] == a[low]` iken sıfıra

bölünmez, eleman yoksa `-1` döner. Karedeki yapıda `low == high` olduğunda payda sıfır olur. Ayrıca aranan yoksa son `mid` döndürülür.

```
int interpolasyon_ara(int a[], int size, int key)
{
    int low = 0, high = size - 1, mid;

    while (low <= high && key >= a[low] && key <= a[high]) {
        if (a[high] == a[low])
            mid = low;
        else
            mid = low + (int)((double)(high - low)
                              / (a[high] - a[low]) * (key - a[low]));
        if (a[mid] == key)
            return mid;
        if (a[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}
```

Bu sürüm dersteki sıralı dizide 14 için indis 5, 88 için -1 döndürdü. 14'ün aranışı adım adım şöyledir:

Adım	`low`	`high`	Hesap	`mid`	`a[mid]`	Karar
1	0	9	$0+\frac{9}{47}\cdot 7 = 1{,}34$	1	8	$8 < 14$, `low = 2`
2	2	9	$2+\frac{7}{45}\cdot 5 = 2{,}78$	2	9	$9 < 14$, `low = 3`
3	3	9	$3+\frac{6}{43}\cdot 3 = 3{,}42$	3	11	$11 < 14$, `low = 4`
4	4	9	$4+\frac{5}{41}\cdot 1 = 4{,}12$	4	13	$13 < 14$, `low = 5`
5	5	9	$5+\frac{4}{40}\cdot 0 = 5$	5	14	Bulundu

Ders sonrası not: Bu dizide interpolasyon 14'ü 5 adımda bulur, 6. bölümdeki ikili arama ise 3 adımda. Sebebi sondaki 54'tür: Dizi düzgün dağılmış değildir ve tahmin her adımda sola kayar. Bu, 8. bölümdeki düzgün dağılım şartının neden gerektiğini gösterir (python ve C ile doğrulandı).

6. İkili Aramanın do-while Sürümü

Hoca, kod sadeleşince ikili aramaya döndü ve `do-while` sürümünü yeniden kurdu. `low` ve `high` başlangıç değerleri döngüden önce atanır, bunlar döngüye alınamaz. Orta nokta hesabı ise `do` içine girer. Döngünün şartı iki parçalıdır: `low <= high` ve bulunamadığı sürece, yani `a[mid] != key`. Sayaç `i` sıfırdan başlar ve her turda bir artar.

Soru: `low = mid + 1` ve `high = mid - 1` yazıyoruz. Buradaki $+1$ ve -1'i neden yazıyoruz?

Cevap: `mid` noktasındaki elemana zaten bakıldı. Ona tekrar bakmamak için büyükse bir sağından, küçükse bir solundan itibaren bakılır.

Hoca karşılaştırmayı önce ters yazdı. Bir öğrenci uyarınca düzeltti: `key` ortadaki elemandan büyükse `low`, küçükse `high` değişir. Eşitlik için `else` yazılmaz, çünkü eşitse hiçbir şey yapılmadan `while` satırına gelinir ve şarttan çıkılır. Kodun çekirdeği şöyledir:

```
do {
    mid = (low + high) / 2;
    i++;
    if (key > a[mid])
        low = mid + 1;
    else if (key < a[mid])
        high = mid - 1;
} while (low <= high && a[mid] != key);
```

Bu sürümle dersteki dizide iki arama yapıldı. Ara değerler hocanın anlatımından kurulmuştur:

- **14 aranır (ara adımlar eklendi):**
  - `low = 0`, `high = 9`, $mid = 4$, `a[4] = 13`. $14 > 13$ olduğu için `low = 5`.
  - `low = 5`, `high = 9`, $mid = 7$, `a[7] = 19`. $14 < 19$ olduğu için `high = 6`.
  - `low = 5`, `high = 6`, $mid = 5$, `a[5] = 14`. Bulundu. Sayaç 3, indis 5.
- **13 aranır:** Hoca "0 ile 9'un ortası 4,5, yani 4'e bakar" dedi. `a[4] = 13` olduğu için eleman ilk denemede bulundu. Sayaç 1, indis 4.

Hocanın vurguladığı nokta şuydu: Eleman ilk denemede bulununca döngü devam etmez ve bu, debug ile adım adım izlenebilir.

> **Ders sonrası not:** Döngüden çıkışta sonucun bulunup bulunmadığını anlamak için `a[mid] == key` kontrolü gerekir. Bu kontrolle yazılan tam sürüm `gcc -Wall -Wextra -std=c89 -pedantic` ile uyarısız derlendi. Dersteki dizide 14 için 3 adımda indis 5, 13 için 1 adımda indis 4, 88 için 4 adımda -1 sonucu verdi.

7. Dev-C++'ta Hata Ayıklama (Debug)

Debug, programın çalışmasını belli bir yerde durdurup adım adım ilerletmektir. Hoca Dev-C++ 5.9.1 üzerinde gösterdi ve benzer özelliklerin başka geliştirme ortamlarında da bulunduğunu söyledi. Hazırlık ve kullanım adımları şunlardır:

1. Kod tek bir dosya olarak değil, proje olarak açılır: File, New Project, ardından projenin altında yeni bir kaynak kod dosyası.
2. Derleyici seçiminde debug modu seçili olmalıdır. Karede bu seçim "TDM-GCC 4.8.1 64-bit Debug" olarak görünür.
3. Durulacak satırın numarasına tek tıklanır. Satır kırmızıya boyanır.
4. Execute menüsünden Debug seçilir. Program o satıra kadar çalışır.
5. Değişkenler "Add watch" ile izlenir veya fare üzerine getirilerek değerleri görülür.

"Add watch" yalnızca değişkenleri değil ifadeleri de izler. Örneğin `size-1` veya 5. bölümdeki kesrin payı izlenebilir. Dizinin adı izlenirse adresi görünür. Karede bu `a = (int *) 0x23fe20` biçimindedir. `a[i]` izlenirse o elemanın değeri görünür. Satıra henüz gelinmediyse değer okunamaz. Next dendikçe güncellenir.

Program durduktan sonra ilerlemek için kullanılan düğmeler şunlardır:

- **Next line:** Bir sonraki satıra geçer.
- **Into function:** Çağrılan fonksiyonun içine girer. Örneğin `dizi_yazdir` içine girilip orada satır satır ilerlenebilir. `printf`'in içine girmeye izin vermez, çünkü bu yalnızca kendi yazdığınız fonksiyonlar içindir.
- **Skip function:** Fonksiyonun içine girmeden geçer ve bir sonraki adıma gelir.
- **Next instruction:** Örneğin bir `for` döngüsü varsa onun iç kısmını adım adım gezdirir.

> **Hocanın vurgusu:** Debug, özellikle koddaki mantıksal hataları, yani yanlış sonuç elde ettiğiniz durumları tespit ederken çok faydalıdır.

8. Hangi Arama Ne Zaman Kullanılır?

Soru: İnterpolasyon aramasını ve ikili aramayı hangi durumlarda birbirine tercih etmeliyiz?

Cevap: Hoca önce ortak şartı söyledi: İkisinde de dizi sıralı olmalıdır. Sonra iki algoritmanın adım sayısını karşılaştırdı.

8.1 İkili Aramanın Adım Sayısı

İkili arama önce $\frac{n}{2}$ elemanı eler, sonra kalanın yarısını, sonra onun yarısını. Arama uzayı adım adım şöyle küçülür:

$$n, \quad \frac{n}{2}, \quad \frac{n}{4}, \quad \frac{n}{8}, \quad \ldots, \quad \frac{n}{2^i}$$

Bir eleman kalana kadar devam edilir. Bu yüzden $\frac{n}{2^i} = 1$ denir. Buradan $2^i = n$ ve iki tarafın 2 tabanında logaritması alınınca $i = \log_2 n$ bulunur. En kötü ihtimalle adım sayısı $\log_2 n$'dir.

Hocanın örneği: 1024 elemanlı bir dizi 2^{10} eleman demektir. En kötü ihtimalle 10 denemede, bir de "artı 1" seçeneğiyle 11 denemede sonuca ulaşılır. Hoca bu artı 1'in algoritma analizi konusu olduğunu ve sonra konuşulacağını söyledi. 1024 elemanda en kötü durumun 11 karşılaştırma olduğu ders sonrasında python ile doğrulandı.

8.2 İnterpolasyon Aramasının Adım Sayısı

İnterpolasyon aramasında adım sayısı için hoca $\log \log n$ değerini verdi. Yani logaritmanın bir kez daha logaritması alınır. Hocanın sayısal örneği $n = 2^{16}$ elemandır:

- **İkili arama:** $\log_2 2^{16} = 16$ denemede bulunur.
- **İnterpolasyon araması:** $\log_2 2^{16} = 16$, ardından $\log_2 16 = 4$. Yani 4 denemede bulunur.

8.3 Düzgün Dağılım Şartı

Bu kazancın bir bedeli vardır. İki algoritma da sıralı dizi ister, ama interpolasyon araması ek olarak düzgün dağılım (uniform distribution) ister. Hoca bunu, dizideki her elemanın sayısının, frekansının birbirine eşit olması diye tarif etti.

Hocanın bir örneği kitap sayfalarıydı. Birinci sayfadan beş yüzüncü sayfaya kadar bütün sayılar birer tane ise interpolasyon araması çalışır, ama anlamsızdır: Üç yüzüncü sayfa üç yüzdedir. Bu durumda ne ikili aramaya ne interpolasyona gerek vardır, dizinin o elemanına doğrudan erişilir.

Asıl kullanışlı durum, sayıların 8'den başlayıp ikişer üçer arttığı ve her sayıdan bir tane (veya her birinden ikişer tane) olduğunun bilindiği durumdur. Böyle bir dizide interpolasyon araması adım sayısını çok iyileştirir.

> **Ders sonrası not:** Hoca düzgün dağılımı "her elemanın frekansının eşit olması" olarak tanımladı. Büyük olasılıkla interpolasyon için asıl gereken, değerlerin aralık boyunca eşit aralıklara yakın, yani doğrusala yakın yayılmasıdır. Her değerden kaç tane olduğu tek başına yetmez. Dersteki dizide her değer birer tanedir, ama sondaki 54 yüzünden interpolasyon ikili aramadan daha yavaş kalmıştır (5. bölüm).

9. İki Sıralı Diziyi Birleştirme (Merge)

Problem şudur: Elimizde iki sıralı dizi var, bunları sıralı tek bir dizide birleştirelim. Hocanın örneği:

- `A` = 2, 4, 6, 8, 10
- `B` = 1, 3, 5, 7, 9

Hoca sınıfa en basit çözümü sordu. İlk öneri, iki dizinin aynı indislerine karşılıklı bakmaktı: Önce ikisinin sıfırıncı elemanı, sonra ikisinin birinci elemanı. Hoca bunun yetmediğini gösterdi: Önce 2 ile 1'e, sonra 4 ile 3'e bakılırsa, indisler birlikte ilerlediği için elemanların sırası kurulamaz. Karşılıklı işaretçiler işe yaramaz.

9.1 Birinci Yaklaşım: Art Arda Kopyala ve Sırala

Bir öğrenci en basit çözümü önerdi: İki diziyi art arda üçüncü bir diziye kopyala, sonra normal şekilde sırala. Hoca bunun basit ve kısmen de etkili olduğunu söyledi.

Dinamik bellek henüz görülmediği için, yeterince yer olduğu kabul edilir. Örneğin `A` ve `B` 50, `C` 100 elemanlık tanımlanır ve dolu kısımların 50'den küçük olduğu bilinir. `C` dizisi 2, 4, 6, 8, 10, 1, 3, 5, 7, 9 olur ve sonra sıralanır.

Maliyet şöyle hesaplanır (dizilerin ikisi de n elemanlı):

- `A`'yı kopyalama: n adım
- `B`'yi kopyalama: n adım
- Sıralama (geçen hafta görülen bubble sort): n^2 adım
- Toplam: kabaca $n^2 + 2n$

Hoca n^2'nin sevilmediğini söyledi ve bunu iyileştirmenin yoluna girdi.

> **Ders sonrası not:** Hoca toplamı $n^2 + 2n$ olarak verdi, analiz sırasında da bir kez "$n^2 + n$" dedi. Bubble sort burada $2n$ elemanlık dizi üzerinde çalışır, yani hocanın n^2 ifadesi bu problemde $(2n)^2 = 4n^2$ mertebesine karşılık gelir. Karesel olma sonucu ve hocanın vardığı karşılaştırma değişmez.

9.2 İkinci Yaklaşım: Üç İndisle Kopyalama

Diğer öğrencilerin önerisini hoca şöyle toparladı: `A` için `i`, `B` için `j`, `C` için `k` adlı bir indis tutulur. `a[i]` ile `b[j]` karşılaştırılır, küçük olan `c[k]`'ya kopyalanır. Küçük olan dizinin indisi bir artar, `k` her seferinde bir artar.

Hoca önce kaba taslak akış diyagramını `for` döngüsüyle çizdi. Adımları şöyledir:

1. Başla. `m` elemanlı `A` ve `n` elemanlı `B` girilmiş kabul edilir.
2. $m + n$ elemanlık `C` ayrılır. `i = 0`, `j = 0` yapılır.
3. `k` sıfırdan başlayıp $m + n$'ye kadar (dahil değil) birer birer artar. Her turda:
    1. `a[i] < b[j]` ise `c[k] = a[i]` ve `i++`.
    2. Değilse `c[k] = b[j]` ve `j++`.

Bir öğrenci, `k++`'nın `for` satırında olduğunu, ayrıca yazılmasına gerek olmadığını hatırlattı. Hoca da doğruladı.

Akış, dizilerin verdiği örnekte (2, 4, 6, 8, 10 ve 1, 3, 5, 7, 9) çalışır: Sırayla 1, 2, 3, …, 10 alınır.

9.3 Bu Yazımdaki Hata ve Sınır Bekçisi

Hoca bu çözüme bir itiraz bekledi ve segmentasyon hatası konusu açıldı. Hoca, bu yazımda segmentasyon hatası oluşacağını söyledi. Şu diziler alınsın:

- `A` = 1, 2, 3, 4, 5, 6
- `B` = 7, 8, 9, 10, 11, 12

Önce 1, 2, 3, 4, 5, 6 kopyalanır. `i` değeri 5 iken `a[5]`, `c[5]`'e yazılır ve `i` 1 artınca 6 olur. Bir sonraki turda program `a[6]` ile karşılaştırma yapmaya çalışır. Oysa `A`'nın son indisi 5'tir, `a[6]` yoktur. Sorun budur.

Bir çözüm, iki dizinin sonuna tam sayının alabileceği en büyük değeri (`INT_MAX`) eklemektir. Böylece `a[6]` kesinlikle vardır, değeri de en büyük olduğu için karşı taraf hep küçük kalır ve program diğer kola gidip kalan elemanları kopyalar.

Soru: `INT_MAX` dediğimiz şey nedir?

Cevap: `limits.h` içindeki bir değerdir. C dili, o bilgisayarda tam sayı kaç bayt ise onun en büyük değerini buraya yerleştirir. Tam sayı bundan büyük olamaz. 4 bayt için bu, $2^{31} - 1$ yani 2147483647'dir (gcc ile doğrulandı).

Hoca bu çözümün bir kusurunu da söyledi: Dizilerin sonuna fazladan değer eklendiği için bellekten kayıp vardır.

> **Ders sonrası not:** Sınır dışı okuma büyük olasılıkla her çalıştırmada segmentasyon hatası vermez. Bellekte o adreste ne varsa onunla karşılaştırma yapıp sessizce yanlış sonuç da üretebilir.

9.4 Daha İyi Çözüm: Kalanları Doğrudan Kopyalamak

Bir öğrenci, `k`'nın her zaman `i + j`'ye eşit olduğunu, bu yüzden `for` döngüsüne gerek kalmadığını ve sınırları kontrol eden bir `while` döngüsünün daha iyi olacağını söyledi. Hoca da bu çözüme geldi.

Temel gözlem şudur: Dizilerden biri kesinlikle diğerinden önce biter. Diziler zaten sıralı olduğu için, biri bitince diğerinin kaldığı yerden sonrası hiç karşılaştırma yapılmadan sona eklenebilir. Hem dizinin sınırı aşılmaz hem de karşılaştırma maliyetinden kurtulunur. Akış diyagramının adımları şöyledir:

1. `A` ve `B` alınır, `C` ayrılır. `i = j = k = 0` yapılır.
2. `i < m` ve `j < n` olduğu sürece:
    1. `a[i] < b[j]` ise `c[k] = a[i]` ve `i++`.
    2. Değilse `c[k] = b[j]` ve `j++`.
    3. `k++`.
3. `i < m` olduğu sürece `c[k] = a[i]`, `k++`, `i++`.
4. `j < n` olduğu sürece `c[k] = b[j]`, `k++`, `j++`.
5. Dizi yazdırılır, dur.

Birinci döngü, dizilerden biri tamamen `C`'ye kopyalanınca biter. Hangisinin bittiği bilinmez. Hoca bunu ayrı bir `if` ile değil, 3. ve 4. adımdaki döngülerin kendi şartıyla çözdü. `i < m` doğruysa `A` bitmemiştir, yani `B` bitmiştir. Yanlışsa `A` bitmiştir ve `B` kopyalanır.

Soru: Hiçbir dizi bitmemiş olamaz mı?

Cevap: Olamaz. Birinci döngüden çıkılmasının tek şartı, `i < m` ve `j < n` şartının bozulmasıdır. Bu da dizilerden birinin bittiği anlamına gelir.

Hocanın ilk örneği için ayrıntılı iz şöyledir (ara adım eklendi):

Tur	`a[i]`	`b[j]`	Kopyalanan	Sonraki `i`	Sonraki `j`	Sonraki `k`
1	2	1	1	0	1	1
2	2	3	2	1	1	2
3	4	3	3	1	2	3
4	4	5	4	2	2	4
5	6	5	5	2	3	5
6	6	7	6	3	3	6
7	8	7	7	3	4	7
8	8	9	8	4	4	8
9	10	9	9	4	5	9

9. turdan sonra $j = 5 = n$ olur ve birinci döngü biter. `A`'da 10 kalmıştır. 3. adımdaki döngü `c[9] = 10` yazar ve $k = 10$ olur. Sonuç 1, 2, 3, ..., 10'dur (python ve C ile doğrulandı).

9.5 Algoritmanın Analizi

Hoca $m = n$ kabul ederek üç durumu inceledi. Diziler kareye göre şunlardır:

Durum	`A`	`B`	Birinci döngü	`i < m` döngüsü	`j < n` döngüsü	Toplam
1	1, 3, 5, 7	2, 4, 6, 8	$2n - 1 = 7$	0	1	$2n = 8$
2	1, 2, 3, 4	5, 6, 7, 8	$n = 4$	0	$n = 4$	$2n = 8$
3	1, 2, 4, 8	3, 5, 6, 7	$2n - 1 = 7$	1	0	$2n = 8$

Her üç durumda toplam $2n$ çıkar. Diziler farklı uzunlukta olsaydı bu $m + n$ olurdu. Böylece sıralamadan kurtulunarak n^2'lik maliyetten $2n$'ye inilmiş olur.

> **Ders sonrası not:** Hoca birinci durum için birinci döngünün $2n$ kez, kalan döngülerin sıfır kez döndüğünü söyledi. Doğrusu, birinci döngü $2n - 1 = 7$ kez döner:

7 kopyalanınca `A` biter ve 8'i kalan döngü kopyalar. Toplam yine $2n$'dir (python ile doğrulandı). Yukarıdaki tablo bu düzeltmeyle yazıldı.

Hocanın vurgusu: Bu önemli bir algoritmadır. Hem zaman açısından önemlidir hem de ileride merge sort diye bir algoritma göreceksiniz. Özellikle Veri Yapıları ve Algoritma Analizi derslerinde farklı biçimlerde karşınıza çıkar. Kodu birçok yerde karşınıza çıkacak.

10. Merge Algoritmasının C Kodu

Hoca algoritmayı yeni bir dosyada kodladı. Diziler önce 50 elemanlı düşünüldü, sonra `#define MAX` ile boyut sabite bağlandı. Ekran karesinde `MAX` 10'dur.

10.1 Dizilerin Doldurulması

Dizilere elle değer girmemek için rastgele sayı üretildi. Bunun için `time.h` eklendi ve `srand(time(NULL))` yazıldı, elemanlar `rand()%MAX` ile dolduruldu. Hoca detayların sonra anlatılacağını, şimdilik sorulmamasını istedi. Merge'in şartı sıralı dizi olduğu için ardından iki dizi `bubble_sirala` ile sıralandı. Hoca, dizilerin fonksiyona referansla çağırma mantığıyla gittiğini hatırlattı.

10.2 dizi_yazdir'a Başlık Parametresi

Hoca çıktıyı okunaklı yapmak için `dizi_yazdir`'a üçüncü bir parametre ekledi: "Before Sort" gibi bir başlık. Kural şudur: Çağrıya üçüncü bir argüman eklendiyse, fonksiyonun tanımına da üçüncü bir parametre yazılmalıdır. Karakter dizisi `char ch[]` olarak tanımlandı ve `printf`'te `%s` ile yazıldı.

Hoca o ana kadarki kodu derleyip denedi. Gerekçesi yine aynıydı: İşin sonunda denerseniz zaman kalmaz. Çıkan uyarıların prototiplerle ilgili olduğunu söyledi ve prototipleri yukarı aldı. Sıralanan dizilerden biri 0, 1, 1, 2, 2, 4, 5, 6, 8, 8 çıktı.

10.3 merge Fonksiyonu

Fonksiyon önce `void` tanımlandı, sonradan `int` yapıldı. Parametrelerin adı `m` ve `n`'dir. Bu fonksiyonda hoca `c[k] = a[i++];` kullandı ve anlamını açıkladı: Önce `a[i]` atanır, sonra `i` artar. Akış diyagramında ise bu birleşik yazımı "kötü bir alışkanlık" diyerek kullanmamıştı.

Eşitlik durumu için ayrı bir `if` yazılmaz. Karşılaştırmaya eşitliği eklemek yeterlidir, bu yüzden `<=` kullanılır. Hangi dizinin bittiği de ayrı bir `if` ile değil, `while`'ın kendi şartıyla anlaşılır. Hocanın ifadesiyle `while`'ın içinde zaten bir `if` vardır, `if`'in içinde ise `while` yoktur.

```c
int merge(int a[], int m, int b[], int n, int c[])
{
    int i = 0, j = 0, k = 0;

    while (i < m && j < n) {
        if (a[i] <= b[j]) {
            c[k] = a[i++];
        }
        else {
            c[k] = b[j++];
        }
        k++;
    }
    while (i < m) {
        c[k++] = a[i++];
    }
    while (j < n) {
        c[k++] = b[j++];
    }
```

```
    return k;
}
```

Kod yazılırken öğrencilerin uyarısıyla üç düzeltme yapıldı. İlk kalan döngüsünün şartı önce `i < n` yazılmış, `i < m` olarak düzeltilmişti. Karede de görüldüğü gibi ikinci kalan döngüsünün şartı önce `j < m` yazılmıştı, `j < n` oldu.

Üçüncü düzeltme bir parametreyle ilgilidir. Hoca ilk başta `C`'nin boyutunu da `l` adında ayrı parametre yapmıştı. Bir öğrenci bu parametrenin kullanılmadığını söyledi. Hoca doğruladı: `k` sıfırdan başlar ve son değeri $m+n$'dir, ayrıca denetlenmesine gerek yoktur. Parametre silindi ve çağrı da tanıma uygun hâle getirildi. Karedeki son prototip şudur: `int merge(int a[], int m, int b[], int n, int c[]);`.

`C` dizisi iki dizinin toplamı kadar yer tutmalıdır. Bu yüzden `C[2*MAX]` olarak tanımlanır.

10.4 Dizi Yazdırmada size Hatası

Birleştirilen dizi yazdırılınca yalnızca ilk 10 elemanı çıktı. Sebep, `dizi_yazdir`'ın içinde döngünün parametreye değil `MAX` sabitine bağlı olmasıydı. Önceki çağrılarda boyut zaten `MAX` olduğu için hata görünmemiş, "denk gelmişti". Döngü `size` parametresine bağlanınca 20 eleman da yazıldı.

Bu sırada hoca, değer verilmemiş `C` dizisinde başlangıçta "çöp değerler" olduğunu gösterdi. Dizinin bazı elemanları sıfır, bazıları başka değerlerdi. Ayrıca birleştirilmiş çıktıda elemanların doğru yerleştiğini, örneğin iki dizideki sıfırların yan yana geldiğini kontrol etti.

10.5 Adım Sayısını Görmek

Hoca analizdeki $2n$'yi çalıştırarak görmek istedi. Önce ayrı bir sayaç (`n`) düşündü, sonra gerek olmadığını fark etti: `k` zaten sıfırdan başlar, birer birer artar ve kaç eleman yazıldığını söyler. Bu yüzden fonksiyon `k` döndürdü ve `main`'de "Adım sayısı" olarak yazdırıldı. 10 + 10 elemanda sonuç 20 çıktı. Hoca rastgele sayıların aralığını 500 ve 600 gibi değerlere çıkardı. Sayılar değişti ama adım sayısı yine 20 kaldı.

> **Ders sonrası not:** `k`, kopyalanan eleman sayısıdır. Döngülerin kuruluşu gereği hep $m+n$ çıkar. Yapılan karşılaştırma sayısı ise verilere göre değişir: 9.5'teki üç durumda sırayla 7, 4 ve 7'dir (python ile doğrulandı). Hocanın ifade ettiği $2n$, kopyalama adımıdır.

10.6 Programın Tamamı

Aşağıdaki kod, kayıt karelerindeki parçalardan ve anlatımdan derlenmiştir. `bubble_sirala`'nın gövdesi önceki dersten alınmıştır. Değişken sırası ve başlık metinleri için aşağıdaki nota bakın.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX 10

void dizi_yazdir(int a[], int size, char ch[]);
void bubble_sirala(int a[], int size);
int merge(int a[], int m, int b[], int n, int c[]);

int main()
{
    int A[MAX], B[MAX], C[2 * MAX];
    int i, adim;

    srand(time(NULL));
    for (i = 0; i < MAX; i++) {
        A[i] = rand() % MAX;
        B[i] = rand() % MAX;
    }
```

```
    dizi_yazdir(A, MAX, "Before Sort\n");
    dizi_yazdir(B, MAX, "Before Sort\n");
    bubble_sirala(A, MAX);
    bubble_sirala(B, MAX);
    dizi_yazdir(A, MAX, "After Sort\n");
    dizi_yazdir(B, MAX, "After Sort\n");
    adim = merge(A, MAX, B, MAX, C);
    dizi_yazdir(C, 2 * MAX, "After Merge\n");
    printf("Adim sayisi = %d\n", adim);
    return 0;
}

void dizi_yazdir(int a[], int size, char ch[])
{
    int i;

    printf("%s", ch);
    for (i = 0; i < size; i++)
        printf("%2d-", a[i]);
    printf("\n");
}

void bubble_sirala(int a[], int size)
{
    int tur, i, temp;

    for (tur = 1; tur < size; tur++)
        for (i = 0; i < size - 1; i++)
            if (a[i] > a[i + 1]) {
                temp = a[i];
                a[i] = a[i + 1];
                a[i + 1] = temp;
            }
}

int merge(int a[], int m, int b[], int n, int c[])
{
    int i = 0, j = 0, k = 0;

    while (i < m && j < n) {
        if (a[i] <= b[j]) {
            c[k] = a[i++];
        }
        else {
            c[k] = b[j++];
        }
        k++;
    }
    while (i < m) {
        c[k++] = a[i++];
    }
    while (j < n) {
        c[k++] = b[j++];
    }
    return k;
}
```

Ders sonrası not: Kod `gcc -Wall -Wextra -std=c89 -pedantic` ile uyarısız derlendi. Çalıştırıldığında “Adim sayisi = 20” yazdı. Karedeki koddan üç fark vardır. `srand` satırı değişken tanımlarının altına alındı (11. bölüm). Karede yalnız `time.h` görünür,

`rand` ve `srand` büyük olasılıkla `stdlib.h`'de bildirildiği için bu başlık eklendi. "After Sort" ve "After Merge" başlıkları da anlatımdan kurulmuştur. Ayrıca `merge` fonksiyonu hocanın ilk örneği olan 2 4 6 8 10 ve 1 3 5 7 9 dizileriyle çalıştırıldı: 1'den 10'a sıralı çıktı ve 10 döndürdü.

11. Değişkenlerin Tanımlanma Yeri

Soru: Bilgisayar Bilimlerine Giriş dersinde, değişkenlerin tipini işlem yapmadan önce fonksiyonun tepesine yazmamız söylenmişti. Burada tanımları `srand`'dan sonra yaptık. Özel bir sebebi var mı?

Cevap: Hoca, `srand`'ın da bir tanımlama, bir makro olduğunu, fonksiyonlarla tanımlamakta sakınca olmadığını söyledi. Diğer hocanın kastettiğini ise ekranda gösterdi: Değişkeni `for` satırının içinde tanımlarsanız (`for (int i = 0; ...)`) C99 standart hatası alırsınız. Bu yüzden "bunu yapmayın" denir.

> **Ders sonrası not:** Hoca `srand` için "makro" dedi. Büyük olasılıkla `srand` bir kütüphane fonksiyonudur ve o satır bir işlemdir. Dolayısıyla öğrencinin kuşkusu haklıdır: Tanımların işlemden sonra gelmesi bu kurala aykırıdır. Ders sonrasında gcc ile denendi. `-std=c89 -pedantic` ile "ISO C90 forbids mixed declarations and code" uyarısı alındı, `-std=gnu89` ile uyarı alınmadı. Hocanın gösterdiği `for` içi tanım ise `-std=gnu89` ile hata verdi ("only allowed in C99 or C11 mode"), `-std=c99` ile derlendi. Bu nedenle `srand` satırını tanımların altına yazmak güvenli olandır.

Soru: `int main` içine `void` yazılabilir mi?

Cevap: Yazılabilir, sorun olmaz. Ama yazmak gerekmez. Önemli olan, kodun CodeBlocks veya Dev-C++'ta derlenmesidir. Hoca aynı uyarıyı tekrarladı: `for` içinde değişken tanımlamayın, yalnızca ilk değer ataması (initialization) yapın. Xcode veya Visual Studio'da bu yazım kabul edilir, ama değişken önceden tanımlanmalıdır.

12. Arama Algoritmalarının Toplu Karşılaştırması

Hoca dersin sonunda üç arama algoritmasını sınıfa sorarak özetledi.

Algoritma	Ön koşul	En iyi durum	En kötü durum (hocanın verdiği)
Sıralı arama (sequential)	Dizi sırasız olabilir	1	n
İkili arama (binary)	Dizi sıralı	1	$\log_2 n$
İnterpolasyon araması	Dizi sıralı ve düzgün dağılmış	1	$\log \log n$

Sıralı arama: İlk bakılan eleman sıfırıncı elemandır. Şanslıysanız aradığınız odur ve bir denemede bulursunuz. En kötü durum sorulduğunda sınıftan "n^2" cevabı da geldi. Hoca doğrusunun n olduğunu söyledi: Bir döngüyle baştan sona bakılır. Eleman hiç yoksa da n deneme boşuna yapılmış olur. Genel karmaşıklık gösterimine (Big-O) bu derste girilmedi.

İkili arama: İlk bakılan eleman ortadaki elemandır. Şanslıysanız ve aradığınız ortadaysa bir denemede bulursunuz. En kötü durumda her seferinde kalan yarıya bakılır ve 8.1'deki türetmeyle $\log_2 n$ çıkar. 1024 elemanda en kötü 10 deneme demektir.

İnterpolasyon araması: En iyi durum yine 1'dir. Hoca bütün şartlar uygun seçilmişse değerin $\log \log n$ olduğunu söyledi.

> **Ders sonrası not:** Hoca $\log \log n$ değerini "worst case" olarak yazdı. Büyük olasılıkla bu, düzgün dağılımda ortalama durumdur. En kötü durumda, örneğin değerler üstel artıyorsa, interpolasyon araması büyük olasılıkla n adıma kadar çıkar. Dersteki dizinin kendisinde bile 5 adım gerekirken ikili arama 3 adımda bulmuştur (5. bölüm). İkili arama için hocanın sonraya bıraktığı "artı 1" ise $\lfloor \log_2 n \rfloor + 1$ biçimindedir, 1024 eleman için 11 eder (python ile doğrulandı).

Hoca ayrıca, burada kullanılanın doğrusal interpolasyon olduğunu söyledi. Orta nokta birinci dereceden değil ikinci dereceden bir fonksiyonla bulunursa kuadratik interpolasyon yapılabilir. Bu, Sayısal Analiz'de interpolasyon konusunda görülecektir.

13. Birinci Laboratuvar Sorusu: Ardışıklığı Bozan Sayı

Kaydın sonunda laboratuvar sorusu anlatılır. Soru metni, kayıt karesindeki belgeden alınmıştır.

Soru metni: Ardışık sayılardan oluşan bir dizide, ardışıklığı bozan kayıp sayıyı bulan algoritmanın kodlanması.

- **Girdi:** Sıralı sayılardan oluşan dizinin boyutu ve dizi elemanları.
- **Çıktı:** Kayıp sayı varsa sayının kendisi, yoksa kayıp sayı olmadığı ekrana yazılmalı.
- **Önemli:** Program büyük boyutlu diziler de düşünülerek verimli şekilde tasarlanmalı.
- **İpucu:** Ardışık sayılar sıralı olduğu için, sıralı bir dizide aramayı kolaylaştıracak yöntemler kullanılmalı.

Girdi	Çıktı
2 3 4 5 7 8 9 10	6
-8 -7 -6 -5 -3 -2 -1 0	-4
10 11 12	No missing value

Asistan, ilk örnekte 5'ten sonra 6 gelmesi gerektiğini, ikinci örnekte kayıp sayının -4 olduğunu, üçüncü örnekte kayıp sayı olmadığını açıkladı. Beklenen yaklaşım, diziye baştan başlayıp tek tek bakmak değildir. Çok büyük bir dizi düşünülmelidir ve bu büyüklükte verimli çalışan bir yöntem kullanılmalıdır. Dersin başında konuşulan "iyi algoritma" ölçütleri ve son derste görülen arama algoritmaları dikkate alınmalıdır.

> **Hocanın vurgusu:** Aklınıza gelen ilk yöntem iyi bir yöntem olmayabilir. Dizinin boyutu küçük diye kolay yöntemlere kaçmayın.

Öğrencilerin soruları ve cevapları şöyledir:

Soru: Dizinin boyutunu biz mi gireceğiz?

Cevap: Evet. Test için küçük boyut girilebilir, ama yöntem çok büyük dizide de hızlı çalışacak gibi düşünülmelidir.

Soru: Boyut `#define MAX` ile sabitlenebilir mi? 1 milyon yazarsam 1 milyon sayı mı gireceğim?

Cevap: Hayır. `MAX`, en büyük dizi boyutunu belirler. Bu kadar yer ayırıp 50 veya 100 elemana kadar kullanılabilir.

Soru: Elemanları kim giriyor?

Cevap: Kullanıcıdan alınır. Girdi alınırken kontrol etmek uygun görülmedi. Önce dizinin elemanları verilir.

Soru: Birden fazla kayıp sayı olabilir mi? Hangisi gösterilmeli?

Cevap: Olabilir. Algoritmanın ilk bulduğu sayıyı göstermek yeterlidir. Bunun soldan ilk sayı olması gerekmez. Örneğin ortadan başlayan bir yöntem önce oradaki kaybı görebilir. Genelde tek kayıp sayı gibi düşünülebilir. Örnekteki dizide 8 de olmasaydı veya 9 da olmasaydı, soldan bakan bir çözüm yine 6'yı gösterecekti.

Soru: Ardışıklık ikişer ikişer de olabilir mi? Dizi azalan olabilir mi?

Cevap: Hayır. Sayılar hep birer birer artar. Dizi negatif sayılardan veya pozitif sayıların ortasından başlayabilir (10, 11, 12 gibi), ama azalan sırada olmaz.

> **Ek bilgi (derste anlatılmadı):** Hoca, sorunun fikrinin ikili arama mantığı olduğunu söylemişti, ama çözüm derste yazılmadı. Aşağıdaki çözüm ders sonrasında hazırlanmıştır. Ardışıklık bozulmamışsa her `i` için `a[i] - a[0] == i` olur. Bunun ilk bozulduğu indis ikili aramayla bulunur ve kayıp sayı `a[0] + low`'dur. Laboratuvar örneklerinde 6, -4 ve "No missing value" verdi, `gcc -Wall -Wextra -std=c89 -pedantic` ile uyarısız derlendi; işlem sayısı büyük olasılıkla $\log_2 n$ mertebesindedir.

```c
#include <stdio.h>
#define MAX 100

int main()
{
    int a[MAX], n, i, low, high, mid;

    scanf("%d", &n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    if (a[n - 1] - a[0] == n - 1) {
        printf("No missing value\n");
    }
    else {
        low = 0;
        high = n - 1;
        while (low < high) {
            mid = (low + high) / 2;
            if (a[mid] - a[0] == mid)
                low = mid + 1;
            else
                high = mid;
        }
        printf("%d\n", a[0] + low);
    }
    return 0;
}
```

Dersin Toparlaması

Bu derste aynı problem için farklı maliyetli çözümler üzerinden gidildi. Palindromlarda yazdırmak ile saymanın ayrı problemler olduğu görüldü: Yazdırmak için genel çözüm tek bir döngüyle bütün sayıları gezmek, saymak için ise tek bir formüldür. Aramada sıralı aramanın n, ikili aramanın $\log_2 n$, interpolasyon aramasının $\log \log n$ değerleri karşılaştırıldı ve ön koşullar eklendikçe adım sayısının düştüğü görüldü. İnterpolasyon kodunun canlı olarak çalışmaması da debug aracının nasıl kullanılacağını gösterdi. Merge algoritmasıyla $n^2 + 2n$ maliyetinden $2n$ maliyetine inildi. Kalan elemanları karşılaştırmadan kopyalama fikri, hem sınır dışı erişimi önledi hem de sınır bekçisi için ayrılan belleğe gerek bırakmadı.

Hoca, kodları kopyalanıp yapıştırılmasın, yazan kişi aynı kodu yeniden yazıp hata yapsın ve düzeltsin diye ekran görüntüsü olarak paylaştığını söyledi. Sonraki derse akademik bağlantı olarak şunlar kaldı: Rastgele sayı üretiminin ayrıntıları, bir dizinin sıralı olup olmadığını kontrol etme ve pointer'lar. Hoca interpolasyon formülünü de tekrar kontrol edeceğini söyledi. Merge ise ileride merge sort'un temeli olarak karşınıza çıkacaktır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.